



Data Structures - CS 212t

Project

2024 – 1446

First Semester

EVENT TICKET BOOKING SYSTEM

Section: 3W7

Student Name	Student ID	Student E-mail
Fajr Omar ALrubeaan	445008461	445008461@pnu.edu.sa
Shujon Ali ALShehri	445008413	445008413@pnu.edu.sa
Renad Fahad Aldoji	445008511	445008511@pnu.edu.sa
Ghadi Mohammad Aldosari	445008427	445008427@pnu.edu.sa
Ghedaf Ahmad Arefein	445008482	445008482@pnu.edu.sa

Table of Contents

I	Project Introduction	2
I.1.1	Overview	2
I.1.2	Problem and solution	2
I.1.3	Solution justification	2
II	Project class diagram	3
III	Concepts covered	4
III.1	Singly Linked List (Customer Management)	4
III.1.1	Method(showCustomers):.....	4
III.1.2	Method(addCustomer):	4
III.1.3	Method(removeCustomers):.....	5
III.1.4	Method(Search):	5
III.2	Queue (Ticket Request Management)	6
III.2.1	Method(Display):.....	6
III.2.2	Method(findTicketByNumber):	6
III.2.3	Method(cancelTicket):	7
III.2.4	Method(countTicketsByEvent):.....	7
III.3	Stack (Booking Management).....	8
III.3.1	Method(display):	8
III.3.2	Method(displayUserBookingHistory):	8
III.3.3	Method(findBookingByID):.....	9
III.3.4	Method(findMostBookedEvent):	9
IV	Main Class.....	10
V	OUTPUT	18
VI	Work distribution	24

I Project Introduction

I.1.1 Overview

The project focuses on developing an **Event Ticket Booking System** that allows users to reserve and purchase tickets for various events, such as sports, and movies. This system will utilize Java programming, incorporating various data structures to enhance functionality and efficiency. By simulating a real-time application, the project aims to deepen our understanding of data structures and their practical applications in software development.

I.1.2 Problem and solution

The primary problem addressed by this project is the challenge of efficiently managing ticket reservations in a high-demand environment. Traditional methods often lead to overbooking, confusion, and delays. **Our solution** is to implement a Java-based application that utilizes data structures such as stacks, queues and linked lists to manage ticket availability and user requests effectively. The queue will handle incoming ticket requests in a first-come, first-served manner, ensuring fairness, while linked lists will be used to store and manage the details of customer information dynamically. This approach not only streamlines the booking process but also enhances user experience by providing quick access to real-time data.

I.1.3 Solution justification

➤ Stack

The stack data structure is employed to **manage recent booking activities**, using the Last-In-First-Out (LIFO) principle. This structure is suitable for handling reversals, such as recent booking cancellations, enabling the system to access and revert the latest bookings if needed. This functionality adds flexibility to the booking process by enabling quick adjustments to the most recent bookings.

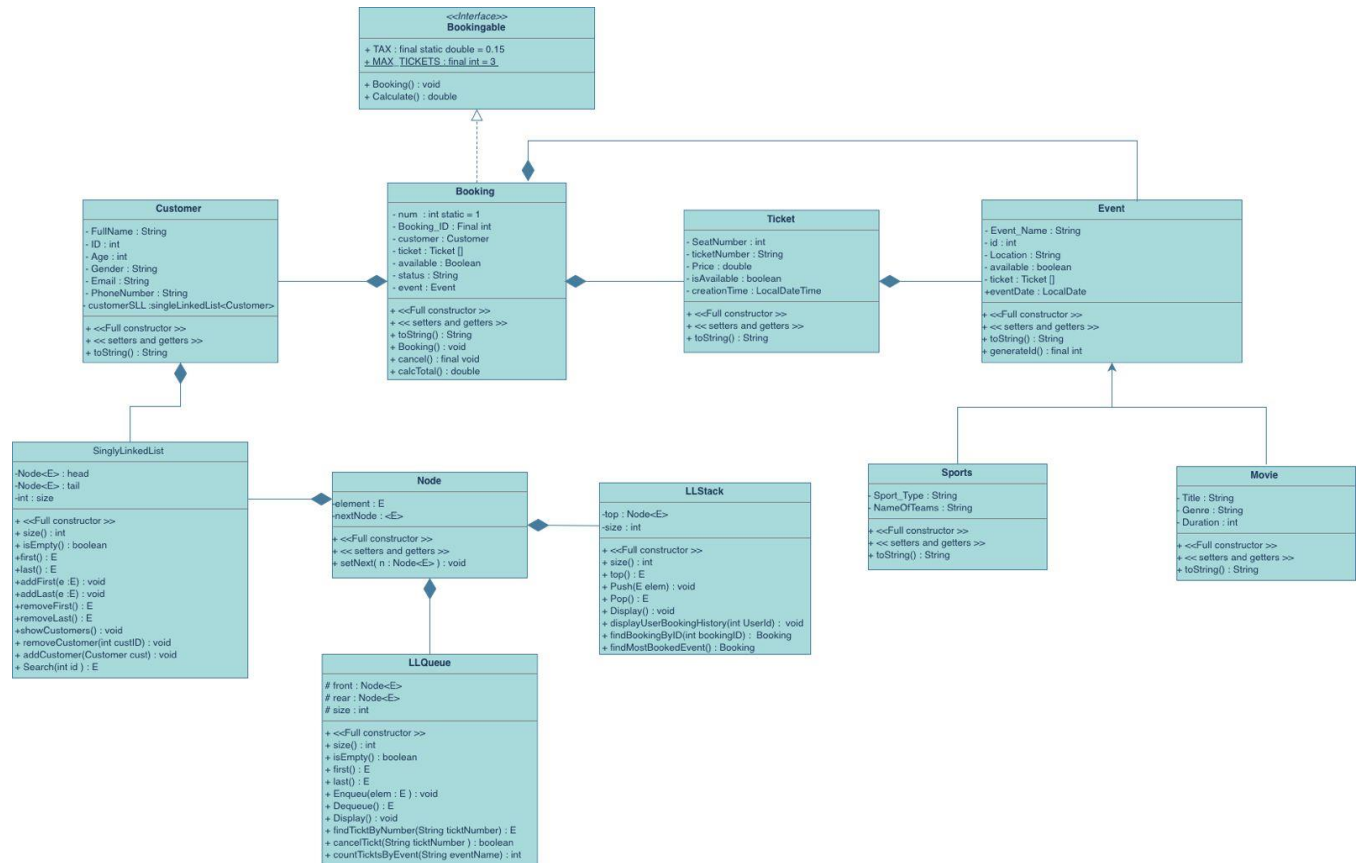
➤ Queue

A queue is utilized to handle **customer ticket requests** in a “first-come, first-served “ manner, following the First-In-First-Out (FIFO) principle. This ensures that all ticket requests are processed in the order they arrive, providing a fair and structured approach to managing customer requests. The queue is particularly effective for maintaining order in ticketing and reducing the likelihood of overbooking or mismanagement of bookings.

➤ Singly linked list

The singly linked list **is used to store and manage customer information**. Each node in the list holds customer details, allowing for efficient insertion and deletion, which is crucial in handling dynamic customer data as bookings are made, modified, or canceled. The linked list's dynamic memory allocation is beneficial for handling a varying number of customer records without wasting memory, supporting the system's flexibility.

II Project class diagram



III Concepts covered

We covered Single LinkedList and Queue and Stack.

```
public class Main {  
  
    public static SinglyLinkedList<Customer> custList = new SinglyLinkedList<>();  
    public static LLQueue<Ticket> TQueue = new LLQueue<>();  
    public static LLStack<Booking> BStack = new LLStack<>();  
    public static ArrayList<Event> eventList = new ArrayList<>();  
    public static Scanner in = new Scanner(source: System.in);  
  
    public static void main(String[] args) {  
        AvailableEvent();  
    }  
}
```

III.1 Singly Linked List (Customer Management)

III.1.1 Method(showCustomers):

The **showCustomers** method traverses the linked list, starting from the head node, and prints out each customer's information. This method is useful for displaying all current customers in the list, providing a quick overview of stored data. It stops when it reaches the end of the list, where the next pointer is null.

```
117  
118 //display method  
119 public void showCustomers(){  
120     Node<E> current = head;  
121     while (current != null) {  
122         System.out.println(" " + current.getElement());  
123         current = current.getNext();  
124     }  
125     System.out.println();  
126 }  
127
```

III.1.2 Method(addCustomer):

This method adds a new customer to the end of the linked list. It creates a new node containing the customer's details and links it as the last node in the list. By using the addLast function, the method efficiently appends new customers, ensuring that the customer data is stored in a straightforward, sequential order.

```
145  
146 //second member method  
147 public void addCustomer(Customer cust){  
148     addLast((E) cust);  
149     System.out.println("\u001B[44m" + "Customer added successfully!");  
150 }  
151  
152 //third member method
```

III.1.3 Method(removeCustomers):

The **removeCustomers** method removes a customer based on a unique customer ID (**custID**). It first checks if the list is empty, then iterates through each customer, removing the first node each time and checking if it matches the given **custID**. If the customer ID doesn't match, the node is added back to the end of the list. This approach ensures that only the specified customer is removed, while other nodes remain intact.

```
127
128 //first member method
129 public void removeCustomer(int custID){
130     if(isEmpty()){
131         System.out.println(x: "No customers available");
132         return;
133     }
134
135     int s = size();
136
137     for (int i=0;i<s;i++){
138         E elm = removeFirst();
139         if(((Customer) elm).getID() != custID) {
140             addLast(e: elm);
141         }
142         System.out.println("\u001B[44m"+"Customer removed successfully");
143     }
144 }
145
```

III.1.4 Method(Search):

The **Search method** searches for a customer based on a **customer ID (id)**. It iterates through each node, starting from the head, and checks if the **customer ID** matches the specified id. If a match is found, it returns the customer's details; if not, it continues until reaching the end of the list. This method is essential for retrieving specific customer information based on a unique identifier.

```
151
152 //third member method
153 public E Sreach(int id) {
154     if (isEmpty()) {
155         System.out.println(x: "No customers available");
156         return null;
157     }
158
159     Node<E> temp = head;
160
161     // Iterate through the list to find the Customer with the given ID
162     while (temp != null) {
163         E elem = temp.getElement();
164
165         // Check if element is a Customer and has the matching ID
166         if (elem instanceof Customer && ((Customer) elem).getID() == id) {
167             return elem; // Return the Customer if ID matches
168         }
169
170         temp = temp.getNext();
171     }
172
173     System.out.println(x: "Customer not found");
174     return null;
175 }
```

III.2 Queue (Ticket Request Management)

III.2.1 Method(Display):

The **Display method** iterates through each element in the queue and prints it. For each iteration, the method removes the front element using **dequeue()**, displays it, and then re-adds it to the end of the queue with **enqueue()**. This approach ensures the original order of the queue is preserved, allowing a complete view of all current tickets without altering the queue structure.

```
86
87 //display method
88 public void Display() {
89     int s = size();
90
91     for (int i = 0; i < s; i++) {
92         E elem = dequeue();
93         System.out.print(elem + " ");
94         enqueue(elem);
95     }
96     System.out.println();
97 }
98
```

III.2.2 Method(findTicketByNumber):

This method searches for a ticket in the queue based on a unique ticket number (**ticketNumber**). It iterates through each element, removing the front of the queue and checking if it matches ticketNumber. If a match is found, it stores the ticket in result. Each ticket is **re-enqueued** after inspection, so the queue order remains unchanged. This function allows for efficient retrieval of a specific ticket while maintaining queue integrity.

```
98
99 // first member method
100 // Method to find a ticket by number
101 public E findTicketByNumber(String ticketNumber) {
102     E result = null;
103     int s = size();
104
105     for (int i = 0; i < s; i++) {
106         E elem = dequeue();
107
108         // Check if elem is a Ticket and compare its ticket number
109         if (elem instanceof Ticket && ((Ticket) elem).getTicketNumber().equals(ticketNumber))
110             result = elem;
111     }
112     enqueue(elem);
113 }
114 return result; // Return the found ticket or null if not found
115 }
116
117
```

III.2.3 Method(cancelTicket):

The **cancelTicket method** cancels a ticket by its number. It iterates through the queue, checking each ticket against the specified ticketNumber. When it finds a match, it **skips re-enqueuing** that ticket, effectively removing it from the queue. The method returns true if the ticket was found and removed, or false if the ticket wasn't present. This approach ensures that the canceled ticket is no longer in the queue while maintaining the order of other tickets.

```
118 // second member method
119 // Method to cancel a ticket by number
120 public boolean cancelTicket(String ticketNumber) {
121     boolean isCanceled = false;
122     int s = size();
123
124     for (int i = 0; i < s; i++) {
125         E elem = dequeue();
126
127         if (elem instanceof Ticket) {
128             Ticket ticket = (Ticket) elem;
129
130             // Check if ticket number matches and it is available
131             if (ticket.getTicketNumber().equals(ticketNumber)) {
132                 ticket.setIsAvailable(isAvailable: true);
133                 isCanceled = true;
134
135                 continue;
136             }
137
138             // Re-enqueue the element if it's not the target ticket or already canceled
139             enqueue(elem);
140         }
141     }
142
143     return isCanceled; // Return
144 }
145
```

III.2.4 Method(countTicketsByEvent):

This method counts how many tickets in the queue are associated with a specific event, identified by eventName. It checks each ticket to see if it contains the eventName (assuming that the element's **toString()** method provides event details). Each ticket is **re-enqueued** after inspection, so the queue's order is preserved. This method is useful for determining event popularity or availability based on ticket requests.

```
146 // third member method
147 // Method to count tickets by event (assumes E has a method getEventName())
148 public int countTicketsByEvent(String eventName) {
149     int count = 0;
150     int s = size();
151
152     for (int i = 0; i < s; i++) {
153         E elem = dequeue();
154         if (elem.toString().contains(eventName)) { // Replace with appropriate method
155             count++;
156         }
157         enqueue(elem);
158     }
159     return count;
160 }
161
```

III.3 Stack (Booking Management)

III.3.1 Method(display):

This method traverses the stack and prints all elements. It pops each element, prints it, and pushes it onto a temporary stack to preserve the original order. After all elements are displayed, the method restores the stack by moving elements back from the temporary stack.

```
70
71 // display method
72 public void Display() {
73     LLStack<E> temp = new LLStack<>();
74     while (!isEmpty()) {
75         E elem = pop();
76         System.out.println(elem+"");
77         temp.push(elem);
78     }
79     System.out.println();
80
81     while (!temp.isEmpty()) {
82         push(elem: temp.pop());
83     }
84 }
```

III.3.2 Method(displayUserBookingHistory):

This method displays the booking history for a specific user, identified by their **userId**. As it pops elements from the stack, it checks if each booking belongs to the specified user by comparing the **user ID**. Matching bookings are displayed. Like the Display method, it uses a temporary stack to preserve the original stack order after processing, ensuring no data is lost or altered.

```
85
86 // first member method display only booking for specific user
87 public void displayUserBookingHistory(int userId) {
88     LLStack<E> temp = new LLStack<>();
89     while (!isEmpty()) {
90         E elem = pop();
91         if (elem instanceof Booking && ((Booking) elem).getCustomer().getID() == userId) {
92             System.out.println(x: elem);
93         }
94         temp.push(elem);
95     }
96
97     while (!temp.isEmpty()) {
98         push(elem: temp.pop());
99     }
100 }
101
102
```

III.3.3 Method(findBookingByID):

This method searches for a booking based on a unique (**bookingID**). It traverses the stack, checking each booking element's ID until it finds a match. Once the specified booking is located, it breaks out of the loop and returns the booking. A temporary stack is used to hold elements during the search, ensuring the stack's original order is maintained after the search is complete. This approach is efficient for locating specific bookings without permanently altering the stack.

```
106 // second member method
107 public Booking findBookingByID(int bookingID) {
108     LLStack<E> tempStack = new LLStack<>();
109     Booking foundBooking = null;
110
111     while (!isEmpty()) {
112         E elem = pop();
113         if (elem instanceof Booking) {
114             Booking currentBooking = (Booking) elem;
115             if (currentBooking.getBooking_ID() == bookingID) {
116                 foundBooking = currentBooking;
117             }
118         }
119         tempStack.push(elem);
120         if (foundBooking != null) {
121             break;
122         }
123     }
124
125     while (!tempStack.isEmpty()) {
126         push(elem: tempStack.pop());
127     }
128
129     return foundBooking;
130 }
131
```

III.3.4 Method(findMostBookedEvent):

This method identifies the most frequently booked event in the stack. For each booking popped from the stack, it compares it with other bookings in the stack to count occurrences. It uses a nested loop, temporarily storing elements in another stack during the comparison, to preserve the stack's original order. After counting occurrences, the method updates the most booked event if the current booking has a higher count than the previously tracked maximum. Once all bookings are processed, the stack is restored, and the most booked event is returned. This method is valuable for determining which event has the highest demand..

```
130 // third member method
131 public Booking findMostBookedEvent() {
132     LLStack<Booking> tempStack = new LLStack<>();
133     Booking mostBooked = null;
134     int maxCount = 0;
135
136     // Outer loop to pick each booking
137     while (!isEmpty()) {
138         Booking currentBooking = (Booking) this.pop();
139         int currentCount = 1;
140
141         // Inner loop to count occurrences of the current booking
142         LLStack<Booking> comparisonStack = new LLStack<>();
143         while (!isEmpty()) {
144             Booking comparisonBooking = (Booking) this.pop();
145             if (currentBooking.equals(obj: comparisonBooking)) {
146                 currentCount++;
147             }
148             comparisonStack.push(elem: comparisonBooking);
149         }
150
151         while (!comparisonStack.isEmpty()) {
152             push((E) comparisonStack.pop());
153         }
154
155         // Update the most booked event
156         if (currentCount > maxCount) {
157             maxCount = currentCount;
158             mostBooked = currentBooking;
159         }
160
161         tempStack.push(elem: currentBooking); // Preserve order
162     }
163
164     while (!tempStack.isEmpty()) {
165         push((E) tempStack.pop());
166     }
167
168     return mostBooked;
169 }
170
171
172
173
```

IV Main Class

❖ Choices (1-7, 0):

- 1. **Add New Customer:** Calls *AddCustomer()* to add customer details.
- 2. **Reserve Booking:** Calls *reserveBooking()* to allow the user to book tickets.
- 3. **Cancel Booking or Ticket:** Asks user to choose between canceling a booking (*cancelBooking()*) or a ticket (*cancelTicket()*).
- 4. **Display All Bookings:** Displays all bookings for a given customer ID via *displayAllBookings()*.
- 5. **Find Booking or Ticket:** Asks user to choose whether to search by booking ID (*findBookingById()*) or ticket number (*findTicketByNumber()*).
- 6. **View Most Booked Event:** Displays the most booked event using *viewMostBookedEvent()*.
- 7. **Modify/View Customer:** Allows modification or removal of customer data using *modifyCustomer()*.
- 0. **Exit:** Ends the program with a message.

```
44         case 4:
45             displayAllBookings();
46             break;
47         case 5:
48             System.out.println("\n Do you want to Search for a Booking or a Ticket?");
49             char find = in.next().charAt(index: 0);
50             switch (find) {
51                 case 'B':
52                     case 'b':
53                         findBookingById();
54                         break;
55                 case 'T':
56                     case 't':
57                         findTicketByNumber();
58                         break;
59             }
60             break;
61         case 6:
62             viewMostBookedEvent();
63             break;
64         case 7:
65             modifyCustomer();
66             break;
67         case 0:
68             System.out.println("\n Thank You!!");
69             running = false;
70             break;
71         default:
72             System.out.println("\u001B[41m" + "Invalid choice. Please try again.");
73     }
74 }
75 }
76 }

1 package Group4_3w7;
2
3 import java.time.LocalDate;
4 import java.util.*;
5
6 public class Main {
7
8     public static SinglyLinkedList<Customer> custList = new SinglyLinkedList<>();
9     public static LLQueue<Ticket> TQueue = new LLQueue<>();
10    public static LLStack<Booking> BStack = new LLStack<>();
11    public static ArrayList<Event> eventList = new ArrayList<>();
12    public static Scanner in = new Scanner(System.in);
13
14    public static void main(String[] args) {
15        AvailableEvent();
16        boolean running = true;
17
18        while (running) {
19            Menu();
20            int choice = in.nextInt();
21            in.nextLine(); // Consume newline
22
23            switch (choice) {
24                case 1:
25                    AddCustomer();
26                    break;
27                case 2:
28                    reserveBooking();
29                    break;
30                case 3:
31                    System.out.println("\n Do you want to cancel a Booking or a Ticket?");
32                    char cancel = in.next().charAt(index: 0);
33                    switch (cancel) {
34                        case 'B':
35                            case 'b':
36                                cancelBooking();
37                                break;
38                        case 'T':
39                            case 't':
40                                cancelTicket();
41                                break;
42                    }
43                    break;
44            }
45        }
46    }
47 }
```

❖ Menu Method: Displaying Options

- Displays a menu with options for the user to interact with the system.

❖ AddCustomer Method: Adding a New Customer

- Prompts for and collects customer details (e.g., name, ID, age, gender, etc.).
- Adds the customer to the custList using **addCustomer()** method.

❖ reserveBooking Method: Reserving a Ticket

- Prompts for the customer ID and checks if the customer exists.
- Allows the user to select an event (Movie or Sport) and displays available events.
- Allows the user to select seats for booking and handles ticket availability and booking confirmation.
- Adds the booking to BStack and displays the booking details and total price.

```
77 public static void Menu() {
78     System.out.println("\u001B[45m" + "==== Ticket Reservation System ==== \n");
79     + "1. New Customer\n"
80     + "2. Reserve booking \n"
81     + "3. Cancel booking/ticket \n"
82     + "4. Display my bookings \n"
83     + "5. Find booking \n"
84     + "6. View Most Booked Events\n"
85     + "7. Modify/View customer\n"
86     + "0. Exit \n");
87
88     System.out.print(": Enter your choice : ");
89
90 }
91
```

```
92 // 1. Add New Customer
93 public static void AddCustomer() {
94     System.out.print(": Enter Your Full Name: ");
95     String fullName = in.nextLine();
96     System.out.print(": Enter Your ID: ");
97     int id = in.nextInt();
98     System.out.print(": Enter Your Age: ");
99     int age = in.nextInt();
100     in.nextLine(); // Consume newline
101     System.out.print(": Enter Your Gender: ");
102     String gender = in.nextLine();
103     System.out.print(": Enter Your Email: ");
104     String email = in.nextLine();
105     System.out.print(": Enter Your Phone Number: ");
106     String phoneNumber = in.nextLine();
107
108     Customer customer = new Customer(fullName, id, age, gender, email, phoneNumber);
109     custList.addCustomer(cust: customer);
110     System.out.println(x: "");
111 }
112
```

```
113 // 2. Reserve Booking
114 public static void reserveBooking() {
115     System.out.print(": Enter Your Customer ID: ");
116     int custID = in.nextInt();
117     Customer customer = (Customer) custList.Sreach(id: custID);
118
119     if (customer == null) {
120         System.out.println("\u001B[41m" + "Customer not found.");
121         return;
122     }
123
124     System.out.println(x: "Do you want to book a Movie or a Sport event?");
125     char eventTypeChoice = in.next().charAt(index: 0);
126
127     String eventType;
128     switch (eventTypeChoice) {
129         case 'M':
130             case 'm':
131                 eventType = "Movie";
132                 break;
133             case 'S':
134                 case 's':
135                     eventType = "Sport";
136                     break;
137             default:
138                 System.out.println("\u001B[41m" + "Invalid choice. Please enter Movie or Sport.");
139                 return;
140     }
141
142     System.out.println("\nAvailable " + eventType + " Events:");
143     List<Event> filteredEvents = new ArrayList<>();
144     for (Event event : eventList) {
145         if (event.getEvent_Name().equalsIgnoreCase(anotherString:eventType)) {
146             filteredEvents.add(e: event);
147             System.out.println((filteredEvents.size()) + " " + event);
148         }
149     }
150
151     if (filteredEvents.isEmpty()) {
152         System.out.println("\u001B[41m" + "No " + eventType + " events available.");
153         return;
154     }
155 }
```

```

154     }
155
156     System.out.print("\nChoose an event by entering the event number: ");
157     int eventChoice = in.nextInt();
158
159     if (eventChoice < 1 || eventChoice > filteredEvents.size()) {
160         System.out.println("\u001B[41m" + "Invalid event choice.");
161         return;
162     }
163
164     Event chosenEvent = filteredEvents.get(eventChoice - 1);
165     Ticket[] availableTickets = chosenEvent.getTicket();
166
167     System.out.println("Available Seats for " + eventType + " Event:");
168     for (Ticket ticket : availableTickets) {
169         if (ticket.isIsAvailable()) {
170             System.out.println("Seat Number: " + ticket.getSeatNumber() + " - Price: $" + ticket.getPrice());
171         }
172     }
173
174     System.out.print("\nEnter the number of seats to be booked: ");
175     int numSeats = in.nextInt();
176
177     if (numSeats > Bookingable.MAX_TICKETS) {
178         System.out.println("\u001B[41m" + "You cannot book more than " + Bookingable.MAX_TICKETS + " tickets at once.");
179         return;
180     }
181
182     Ticket[] selectedTickets = new Ticket[numSeats];
183     for (int i = 0; i < numSeats; i++) {
184         System.out.print("\nEnter Seat Number to reserve: ");
185         int seatNumber = in.nextInt();
186         boolean seatFound = false;
187
188         for (Ticket ticket : availableTickets) {
189             if (ticket.getSeatNumber() == seatNumber && ticket.isIsAvailable()) {
190                 ticket.setIsAvailable(isAvailable: false); // Mark ticket as reserved
191                 ticket.setTicketNumber(ticketNumber: generateTicketNumber()); // Assign ticket number
192                 selectedTickets[i] = ticket;
193                 seatFound = true;
194                 break;
195             }
196         }
197     }
198
199     if (!seatFound) {
200         System.out.println("Seat number " + seatNumber + " is not available. Please choose another seat.");
201         i--; // Retry current seat selection
202     }
203
204     Booking booking = new Booking(Booking_ID:Booking.getNum(), customer, ticket:selectedTickets, available: true, status:"Confirmed", event: chosenEvent);
205     BStack.push(elem: booking);
206
207     displayBookingDetails(booking);
208
209     System.out.println("");
210     System.out.println("Total Price: $" + booking.calcTotal());
211     booking.Booking();
212 }
213

```

❖ cancelBooking Method: Canceling a Booking

- Prompts for a booking ID and cancels the booking if found. Marks all associated tickets as available again.

❖ cancelTicket Method: Canceling a Ticket

- Prompts for a ticket number and cancels the ticket if found in the ticket queue (TQueue).
- Updates the related booking by removing the canceled ticket from it.

```
214 // 3. Cancel Booking
215 public static void cancelBooking() {
216     System.out.print(s: "Enter Booking ID to cancel: ");
217     int bookingID = In.nextInt();
218     Booking booking = (Booking) BStack.findBookingByID(bookingID);
219
220     if (booking != null) {
221         // Cancel all tickets in the booking
222         Ticket[] tickets = booking.getTicket();
223
224         for (Ticket ticket : tickets) {
225             if (ticket != null) {
226                 ticket.setIsAvailable(isAvailable: true); // Set the ticket as available
227             }
228         }
229
230         booking.cancel();
231     } else {
232         System.out.println(x: "Booking not found.");
233     }
234 }
235
```

```
236 // 3. Cancel Ticket
237 public static void cancelTicket() {
238     System.out.print(s: "Enter the ticket number to cancel: ");
239     String ticketNumber = In.next();
240
241     Ticket foundTicket = TQueue.findTicketByNumber(ticketNumber);
242
243     if (foundTicket == null) {
244         System.out.println("\u001B[41m" + "Ticket with ticket number " + ticketNumber + " was not found.");
245         return;
246     }
247
248     boolean isCanceled = TQueue.cancelTicket(ticketNumber);
249
250     if (isCanceled) {
251         System.out.println("Ticket with ticket number " + ticketNumber + " has been successfully canceled.");
252
253         LLStack<Booking> tempStack = new LLStack<>();
254         Booking relatedBooking = null;
255
256         while (!BStack.isEmpty()) {
257             Booking booking = BStack.pop();
258             boolean ticketFoundInBooking = false;
259
260             // Check if the ticket is part of this booking
261             Ticket[] tickets = booking.getTicket();
262             for (int i = 0; i < tickets.length; i++) {
263                 if (tickets[i] != null && tickets[i].getTicketNumber().equals(anObject: ticketNumber)) {
264                     tickets[i] = null; // Remove the ticket from the booking
265                     ticketFoundInBooking = true;
266                     break;
267                 }
268             }
269
270             if (ticketFoundInBooking) {
271                 relatedBooking = booking;
272             }
273
274             tempStack.push(elem: booking);
275         }
276     }
277
```

```
276 // Restore the original stack state
277 while (!tempStack.isEmpty()) {
278     BStack.push(elem: tempStack.pop());
279 }
280
281 } else {
282     System.out.println("\u001B[41m" + "Ticket with ticket number " + ticketNumber + " was already canceled.");
283 }
284 }
285
286
```

❖ **displayAllBookings Method: Displaying All Bookings for a Customer**

- Prompts for a customer ID and displays all bookings associated with that customer using *displayUserBookingHistory()* from BStack.

❖ **findBookingById Method: Finding a Booking by ID**

- Prompts for a booking ID and displays the booking details if found using *displayBookingDetails()*.

❖ **findTicketByNumber Method: Finding a Ticket by Number**

- Prompts for a ticket number and displays its details (seat number, ticket number, availability).
- Also checks and displays the related booking if the ticket is part of a booking.

```
287 // 4. Display User Bookings
288 public static void displayAllBookings() {
289     System.out.print(s: "Enter your ID: ");
290     int Id = in.nextInt();
291     Customer customer = (Customer) custList.Sreach(id: Id);
292
293     if (customer == null) {
294         System.out.println("\u001B[41m" + "Customer not found.");
295         return;
296     }
297     System.out.println("\nDisplaying bookings for user ID: " + Id);
298     BStack.displayUserBookingHistory(userId: Id);
299 }
300
301 // 5. Find Booking by ID
302 public static void findBookingById() {
303     System.out.print(s: "Enter Booking ID to search: ");
304     int bookingID = in.nextInt();
305     Booking booking = (Booking) BStack.findBookingByID(bookingID);
306
307     // Use the helper method to display the booking and ticket details
308     displayBookingDetails(booking);
309 }
```

```

311 // 5. Find Ticket by Number
312 public static void findTicketByNumber() {
313     System.out.print("\nEnter the ticket number to search: ");
314     String ticketNumber = in.next();
315
316     Ticket foundTicket = TQueue.findTicketByNumber(ticketNumber);
317
318     if (foundTicket != null) {
319         System.out.println("\n== Ticket Details ==");
320         System.out.println("Seat Number: " + foundTicket.getSeatNumber());
321         System.out.println("Ticket Number: " + foundTicket.getTicketNumber());
322         System.out.println("Price: $" + foundTicket.getPrice());
323         System.out.println("Available: " + (foundTicket.isAvailable() ? "Yes" : "No"));
324
325         // Search for the related booking in BStack
326         Booking relatedBooking = null;
327         LLStack<Booking> tempStack = new LLStack<>();
328
329         while (!BStack.isEmpty()) {
330             Booking booking = BStack.pop();
331             for (Ticket bookedTicket : booking.getTicket()) {
332                 if (bookedTicket != null && bookedTicket.getTicketNumber().equals(ticketNumber)) {
333                     relatedBooking = booking;
334                     break;
335                 }
336             }
337             tempStack.push(booking);
338             if (relatedBooking != null) {
339                 break;
340             }
341         }
342
343         // Restore BStack to its original state
344         while (!tempStack.isEmpty()) {
345             BStack.push(tempStack.pop());
346         }
347
348         // Display related booking if found
349         if (relatedBooking != null) {
350             displayBookingDetails(booking: relatedBooking);
351         } else {
352             System.out.println("\nRelated booking not found.");
353         }
354     } else {
355         System.out.println("\u001B[41m" + "Ticket with ticket number " + ticketNumber + " was not found.");
356     }
357 }
358

```

❖ **viewMostBookedEvent Method: Viewing the Most Booked Event**

- Finds the most booked event and displays its details (name, location, date, etc.).
- Prompts the user if they want to book tickets for this event and triggers the **reserveBooking()** method.

❖ **modifyCustomer Method: Modifying or Removing Customer Information**

- Prompts for a customer ID and displays the current customer details.
- Allows the user to modify customer details (name, age, gender, etc.) or remove the customer from the list.

```

// 6. View the most booked event and prompt for reservation
public static void viewMostBookedEvent() {
    Booking mostBookedEvent = BStack.findMostBookedEvent(); // Finds the most booked event

    if (mostBookedEvent != null) {
        Event associatedEvent = mostBookedEvent.getEvent();

        if (associatedEvent != null) {
            System.out.println("\n== Most Booked Event ==");
            System.out.println("Event Name: " + associatedEvent.getEvent_Name());

            if (associatedEvent instanceof Movie) {
                Movie movieEvent = (Movie) associatedEvent; // Cast to Movie
                System.out.println("Movie Title: " + movieEvent.getTitle());
            }

            System.out.println("Location: " + associatedEvent.getLocation());
            System.out.println("Date: " + associatedEvent.getEventDate());

            // Ask the user if they want to book this event
            System.out.print("\nWould you like to book this event? (yes/no): ");
            char response = in.next().charAt(0);

            switch (response) {
                case 'Y':
                case 'y':
                    reserveBooking(); // Call the reserveBooking method
                    break;
                default:
                    System.out.println("\nReturning to the main menu.");
            }
        } else {
            System.out.println("\u001B[41m" + "No event associated with the most booked booking.");
        }
    } else {
        System.out.println("\u001B[41m" + "No events found in booking history.");
    }
}

```

```

400 // 7. View, Modify, Remove customer
401 public static void modifyCustomer() {
402
403     System.out.print(s: "Enter Customer ID: ");
404     int custID = in.nextInt();
405
406     Customer cust = custList.Sreach(id: custID);
407
408     if (cust == null) {
409         // If the customer doesn't exist, return immediately
410         return;
411     }
412
413     System.out.println(s: "\n== Customer Information ==");
414     System.out.println("Full Name: " + cust.getFulName());
415     System.out.println("ID: " + cust.getID());
416     System.out.println("Age: " + cust.getAge());
417     System.out.println("Gender: " + cust.getGender());
418     System.out.println("Email: " + cust.getEmail());
419     System.out.println("Phone Number: " + cust.getPhoneNumber());
420
421     System.out.println(s: "Do you want to modify/remove? (Y/N) ");
422     char answer = in.next().charAt(index: 0);
423
424     if (answer == 'Y' || answer == 'y') {
425
426         // Prompt the user for modification options
427         System.out.println(s: "\nWhat would you like to do?");
428         System.out.println(s: "1. Edit Customer Information");
429         System.out.println(s: "2. Remove Customer");
430         System.out.println(s: "0. Cancel");
431         System.out.print(s: "Enter your choice: ");
432
433         int choice = in.nextInt();
434         in.nextLine(); // Consume newline
435
436         switch (choice) {
437             case 1: // Edit customer information
438                 System.out.println(s: "\n== Edit Customer Information ==");
439
440                 // Allow the user to modify each field if desired
441                 System.out.print(s: "Enter new Full Name (or press Enter to keep current): ");
442                 String fullName = in.nextLine();
443                 if (!fullName.isEmpty()) {
444                     cust.setFulName(fullName);
445                 }
446
447                 System.out.print(s: "Enter new Age (or press Enter to keep current): ");
448                 String ageInput = in.nextLine();
449                 if (!ageInput.isEmpty()) {
450                     cust.setAge(Age: Integer.parseInt(s: ageInput));
451                 }
452
453                 System.out.print(s: "Enter new Gender (or press Enter to keep current): ");
454                 String gender = in.nextLine();
455                 if (!gender.isEmpty()) {
456                     cust.setGender(Gender: gender);
457                 }
458
459                 System.out.print(s: "Enter new Email (or press Enter to keep current): ");
460                 String email = in.nextLine();
461                 if (!email.isEmpty()) {
462                     cust.setEmail(email);
463                 }
464
465                 System.out.print(s: "Enter new Phone Number (or press Enter to keep current): ");
466                 String phoneNumber = in.nextLine();
467                 if (!phoneNumber.isEmpty()) {
468                     cust.setPhoneNumber(phoneNumber);
469                 }
470
471                 System.out.println("\u001B[44m" + "Customer information updated successfully!");
472                 break;
473
474             case 2: // Remove customer
475                 custList.removeCustomer(custID);
476                 break;
477
478             case 0: // Cancel
479                 System.out.println(s: "Operation cancelled.");
480                 break;
481
482             default:
483                 System.out.println("\u001B[41m" + "Invalid choice. Please try again.");
484                 break;
485         }
486     }
487 }

```

❖ generateTicket Method: Generating Tickets for Events

- Generates a specified number of tickets and adds them to the ticket queue (TQueue) with a given price.

❖ AvailableEvent Method: Initializing Events

- Checks if there are any events in eventList, and if not, creates sample events (movies and sports).
- Adds sample bookings with reserved tickets to simulate real bookings for display.

❖ generateTicketNumber Method: Generating a Unique Ticket Number

- Generates a random 6-digit ticket number for a new ticket.

❖ displayBookingDetails Method: Displaying Booking Information

- Displays detailed booking information (booking ID, customer name, ticket details, and total price) for a given booking.

```
488 private static Ticket[] generateTicket(int count, double price) {
489     Ticket[] t = new Ticket[count];
490     for (int i = 0; i < count; i++) {
491         t[i] = new Ticket(i + 1, ticketNumber, generateTicketNumber(), price, isAvailable: true);
492         TQueue.enqueue(t[i]);
493     }
494     return t;
495 }
496
497 private static void AvailableEvent() {
498     if (eventList.isEmpty()) {
499         Ticket[] movie = generateTicket(count: 7, price: 60);
500         Ticket[] sport = generateTicket(count: 5, price: 75);
501
502         // Adding movie events to the list
503         Event busyBee = new Movie(title: "Busy Bee", Genre: "Sci-Fi", Duration: 120, Event_Name: "Movie", id: 1003, Location: "Cinema City", available: true, ticket: movie,
504             EventManager: LocalDate.now().plusDays(daysToAdd: 2));
505         Event manifest = new Movie(title: "Manifest", Genre: "Horror", Duration: 160, Event_Name: "Movie", id: 1004, Location: "Grand Theater", available: true, ticket: movie,
506             EventManager: LocalDate.now().plusDays(daysToAdd: 2));
507         Event basketball = new Sports(sport_Type: "Basketball", NameOfTeams: "Lakers vs Mavericks", Event_Name: "Sport", id: 3007, Location: "PNU Stadium", available: true,
508             ticket: sport, EventManager: LocalDate.now().plusDays(daysToAdd: 3));
509         Event football = new Sports(sport_Type: "Football", NameOfTeams: "Eagles vs Colts", Event_Name: "Sport", id: 3009, Location: "UBS Arena", available: true, ticket: sport,
510             EventManager: LocalDate.now().plusDays(daysToAdd: 4));
511
512         eventList.add(busyBee);
513         eventList.add(manifest);
514         eventList.add(basketball);
515         eventList.add(football);
516
517         // Create fake bookings to show on most booked event
518         Customer fakeCustomer = new Customer(FullName: "", Id: 0, Age: 0, Gender: "", Email: "@example.com", phoneNumbers: "000000");
519
520         Ticket[] reservedTickets = movie[0..movie.length-1];
521         for (Ticket ticket : reservedTickets) {
522             ticket.setIsAvailable(isAvailable: false); // Mark as unavailable
523             ticket.setTicketNumber(ticketNumber: generateTicketNumber());
524         }
525
526         Booking fakeBooking = new Booking(Booking_ID: Booking.getNum(), customer: fakeCustomer, ticket: reservedTickets, available: true, status: "Confirmed", event: manifest);
527         BStack.push(elem: fakeBooking);
528     }
529 }
530
531 private static String generateTicketNumber() {
532     Random rand = new Random();
533     StringBuilder ticketNumber = new StringBuilder();
534     for (int i = 0; i < 6; i++) {
535         ticketNumber.append(i: rand.nextInt(bound: 10)); // Append a random digit
536     }
537     return ticketNumber.toString();
538 }
539
540 private static void displayBookingDetails(Booking booking) {
541     if (booking != null) {
542         // Display detailed booking information
543         System.out.println("\n== Booking Information ==");
544         System.out.println("Booking ID: " + booking.getBooking_ID());
545         System.out.println("Customer Name: " + booking.getCustomer().getFullName());
546         System.out.println("Customer ID: " + booking.getCustomer().getId());
547         System.out.println("Status: " + booking.getStatus());
548         System.out.println("Total Price: $" + booking.calcTotal());
549
550         // Display ticket information for this booking
551         System.out.println("\n== Ticket Information ==");
552         Ticket[] tickets = booking.getTicket();
553         if (tickets != null && tickets.length > 0) {
554             boolean hasValidTickets = false;
555             for (Ticket ticket : tickets) {
556                 if (ticket != null) { // Ensure we only process non-null tickets
557                     hasValidTickets = true;
558                     System.out.println("Seat Number: " + ticket.getSeatNumber()
559                         + ", Ticket Number: " + ticket.getTicketNumber()
560                         + ", Price: $" + ticket.getPrice()
561                         + ", Available: " + (ticket.isAvailable() ? "Yes" : "No"));
562                 }
563             }
564             if (!hasValidTickets) {
565                 System.out.println("\nNo valid tickets found for this booking.");
566             }
567         } else {
568             System.out.println("\nNo tickets found for this booking.");
569         }
570     } else {
571         System.out.println("\u001B[41m" + "Booking not found.");
572     }
573 }
574 }
575 }
```

V OUTPUT

```
run:
=== Ticket Reservation System ===
1. New Customer
2. Reserve booking
3. Cancel booking/ticket
4. Display my bookings
5. Find booking
6. View Most Booked Events
7. Modify/View customer
0. Exit

Enter your choice : 1
Enter Your Full Name: ghedaf are
Enter Your ID: 1145
Enter Your Age: 20
Enter Your Gender: female
Enter Your Email: 0000@example.com
Enter Your Phone Number: 0555555555
Customer added successfully!

=== Ticket Reservation System ===
1. New Customer
2. Reserve booking
3. Cancel booking/ticket
4. Display my bookings
5. Find booking
6. View Most Booked Events
7. Modify/View customer
0. Exit

Enter your choice : 2
Enter Your Customer ID: 1145
Do you want to book a Movie or a Sport event?
sport

Available Sport Events:
1. Sport Type: Basketball, Name Of Teams: Lakers vs Mavericks Location: PNU Stadium, Date: 2024-11-18
2. Sport Type: Football, Name Of Teams: Eagles vs Colts Location: UBS Arena, Date: 2024-11-19

Choose an event by entering the event number: 3
Invalid event choice.

=== Ticket Reservation System ===
1. New Customer
2. Reserve booking
3. Cancel booking/ticket
4. Display my bookings
5. Find booking
6. View Most Booked Events
7. Modify/View customer
0. Exit

Enter your choice : 2
Enter Your Customer ID: 1111
Customer not found
Customer not found.

=== Ticket Reservation System ===
1. New Customer
2. Reserve booking
3. Cancel booking/ticket
4. Display my bookings
5. Find booking
6. View Most Booked Events
7. Modify/View customer
0. Exit

Enter your choice : 2
Enter Your Customer ID: 1145
Do you want to book a Movie or a Sport event?
game
Invalid choice. Please enter Movie or Sport.
```

=== Ticket Reservation System ===

1. New Customer
2. Reserve booking
3. Cancel booking/ticket
4. Display my bookings
5. Find booking
6. View Most Booked Events
7. Modify/View customer
0. Exit

Enter your choice : 2

Enter Your Customer ID: 1145

Do you want to book a Movie or a Sport event?

sport

Available Sport Events:

1. Sport Type: Basketball, Name Of Teams: Lakers vs Mavericks Location: PNU Stadium, Date: 2024-11-18
2. Sport Type: Football, Name Of Teams: Eagles vs Colts Location: UBS Arena, Date: 2024-11-19

Choose an event by entering the event number: 1

Available Seats for Sport Event:

- Seat Number: 1 - Price: \$75.0
Seat Number: 2 - Price: \$75.0
Seat Number: 3 - Price: \$75.0
Seat Number: 4 - Price: \$75.0
Seat Number: 5 - Price: \$75.0

Enter the number of seats to be booked: 4

You cannot book more than 3 tickets at once.

Enter the number of seats to be booked: 2

Enter Seat Number to reserve: 5

Enter Seat Number to reserve: 9

Seat number 9 is not available. Please choose another seat.

Enter Seat Number to reserve: 4

=== Booking Information ===

Booking ID: 2

Customer Name: ghedaf are

Customer ID: 1145

Status: Confirmed

Total Price: \$138.0

=== Ticket Information ===

Seat Number: 5, Ticket Number: 724837, Price: \$60.0, Available: No

Seat Number: 4, Ticket Number: 098234, Price: \$60.0, Available: No

Total Price: \$138.0

booking has been confirmed.

=== Ticket Reservation System ===

1. New Customer
2. Reserve booking
3. Cancel booking/ticket
4. Display my bookings
5. Find booking
6. View Most Booked Events
7. Modify/View customer
0. Exit

Enter your choice : 3

Do you want to cancel a Booking or a Ticket?

ticket

Enter the ticket number to cancel: 724837

Ticket with ticket number 724837 has been successfully canceled.

=== Ticket Reservation System ===

1. New Customer
2. Reserve booking
3. Cancel booking/ticket
4. Display my bookings
5. Find booking
6. View Most Booked Events
7. Modify/View customer
0. Exit

Enter your choice : 3

Do you want to cancel a Booking or a Ticket?

booking

Enter Booking ID to cancel: 2

Booking cancelled successfully.

Two Cases Of Choice 4, one before choice 3 is selected and one after choice 3 is selected, **canceling will change the isAvailable to true.**

```
=== Ticket Reservation System ===
1. New Customer
2. Reserve booking
3. Cancel booking/ticket
4. Display my bookings
5. Find booking
6. View Most Booked Events
7. Modify/View customer
0. Exit

Enter your choice : 4
Enter your ID: 1145

Displaying bookings for user ID: 1145

Booking_ID: 2
-----
Customer:
FullName:ghedaf are
ID:1145
Age:20
Gender:female
Email:0000@example.com
phoneNumber: 0555555555
-----
[Ticket: seatNumber: 5
price: 60.0
isAvailable: false
creationTime: 2024-11-15T20:39:45.639158, Ticket: seatNumber: 4
price: 60.0
isAvailable: false
creationTime: 2024-11-15T20:39:45.639106]
Event: Movie at Grand Theater on 2024-11-17
status: Confirmed
```

```
=== Ticket Reservation System ===
1. New Customer
2. Reserve booking
3. Cancel booking/ticket
4. Display my bookings
5. Find booking
6. View Most Booked Events
7. Modify/View customer
0. Exit

Enter your choice : 4
Enter your ID: 1145

Displaying bookings for user ID: 1145

Booking_ID: 2
-----
Customer:
FullName:ghedaf are
ID:1145
Age:20
Gender:female
Email:0000@example.com
phoneNumber: 0555555555
-----
[Ticket: seatNumber: 5
price: 60.0
isAvailable: true
creationTime: 2024-11-15T20:39:45.639158, Ticket: seatNumber: 4
price: 60.0
isAvailable: true
creationTime: 2024-11-15T20:39:45.639106]
Event: Movie at Grand Theater on 2024-11-17
status: Cancelled
```

```

=== Ticket Reservation System ===
1. New Customer
2. Reserve booking
3. Cancel booking/ticket
4. Display my bookings
5. Find booking
6. View Most Booked Events
7. Modify/View customer
0. Exit

Enter your choice : 5
Do you want to Search for a Booking or a Ticket?
ticket
Enter the ticket number to search: 724837

=== Ticket Details ===
Seat Number: 5
Ticket Number: 724837
Price: $60.0
Available: No

=== Booking Information ===
Booking ID: 2
Customer Name: ghedaf are
Customer ID: 1145
Status: Confirmed
Total Price: $138.0

=== Ticket Information ===
Seat Number: 5, Ticket Number: 724837, Price: $60.0, Available: No
Seat Number: 4, Ticket Number: 098234, Price: $60.0, Available: No

```

```

=== Ticket Reservation System ===
1. New Customer
2. Reserve booking
3. Cancel booking/ticket
4. Display my bookings
5. Find booking
6. View Most Booked Events
7. Modify/View customer
0. Exit

Enter your choice : 5
Do you want to Search for a Booking or a Ticket?
booking
Enter Booking ID to search: 2

=== Booking Information ===
Booking ID: 2
Customer Name: ghedaf are
Customer ID: 1145
Status: Confirmed
Total Price: $138.0

=== Ticket Information ===
Seat Number: 5, Ticket Number: 724837, Price: $60.0, Available: No
Seat Number: 4, Ticket Number: 098234, Price: $60.0, Available: No

```

After **canceling** one of the tickets → **1 ticket remains**

```

=== Ticket Reservation System ===
1. New Customer
2. Reserve booking
3. Cancel booking/ticket
4. Display my bookings
5. Find booking
6. View Most Booked Events
7. Modify/View customer
0. Exit

Enter your choice : 5
Do you want to Search for a Booking or a Ticket?
booking
Enter Booking ID to search: 2

=== Booking Information ===
Booking ID: 2
Customer Name: ghedaf are
Customer ID: 1145
Status: Confirmed
Total Price: $69.0

=== Ticket Information ===
Seat Number: 4, Ticket Number: 098234, Price: $60.0, Available: No

```

=== Ticket Reservation System ===

1. New Customer
2. Reserve booking
3. Cancel booking/ticket
4. Display my bookings
5. Find booking
6. View Most Booked Events
7. Modify/View customer
0. Exit

Enter your choice : 6

=== Most Booked Event ===

Event Name: Movie

Movie Title: Manifest

Location: Grand Theater

Date: 2024-11-17

Would you like to book this event? (yes/no): yes

Enter Your Customer ID: 1145

Do you want to book a Movie or a Sport event?

movie

Available Movie Events:

1. Movie Title: Busy Bee, Genre: Sci-Fi, Duration: 120mins, Location: Cinema City, Date: 2024-11-16
2. Movie Title: Manifest, Genre: Horror, Duration: 168mins, Location: Grand Theater, Date: 2024-11-17

Choose an event by entering the event number: 2

Available Seats for Movie Event:

Seat Number: 3 - Price: \$60.0

Seat Number: 4 - Price: \$60.0

Seat Number: 5 - Price: \$60.0

Seat Number: 6 - Price: \$60.0

Seat Number: 7 - Price: \$60.0

=== Ticket Reservation System ===

1. New Customer
2. Reserve booking
3. Cancel booking/ticket
4. Display my bookings
5. Find booking
6. View Most Booked Events
7. Modify/View customer
0. Exit

Enter your choice : 9

Invalid choice. Please try again.

=== Ticket Reservation System ===

1. New Customer
2. Reserve booking
3. Cancel booking/ticket
4. Display my bookings
5. Find booking
6. View Most Booked Events
7. Modify/View customer
0. Exit

Enter your choice : 0

Thank You!!

BUILD SUCCESSFUL (total time: 6 minutes 47 seconds)

```

=== Ticket Reservation System ===
1. New Customer
2. Reserve booking
3. Cancel booking/ticket
4. Display my bookings
5. Find booking
6. View Most Booked Events
7. Modify/View customer
0. Exit

Enter your choice : 7
Enter Customer ID: 1145

=== Customer Information ===
Full Name: ghedaf are
ID: 1145
Age: 20
Gender: female
Email: 0000@example.com
Phone Number: 0555555555
Do you want to modify/remove? (Y/N)
y

What would you like to do?
1. Edit Customer Information
2. Remove Customer
0. Cancel
Enter your choice: 1

=== Edit Customer Information ===
Enter new Full Name (or press Enter to keep current): Ghedaf Arefein
Enter new Age (or press Enter to keep current):
Enter new Gender (or press Enter to keep current):
Enter new Email (or press Enter to keep current): ghedaf@gmail.com
Enter new Phone Number (or press Enter to keep current):
Customer information updated successfully!

```

```

=== Ticket Reservation System ===
1. New Customer
2. Reserve booking
3. Cancel booking/ticket
4. Display my bookings
5. Find booking
6. View Most Booked Events
7. Modify/View customer
0. Exit

Enter your choice : 7
Enter Customer ID: 1145

=== Customer Information ===
Full Name: ghedaf are
ID: 1145
Age: 20
Gender: female
Email: 0000@example.com
Phone Number: 0555555555
Do you want to modify/remove? (Y/N)
y

What would you like to do?
1. Edit Customer Information
2. Remove Customer
0. Cancel
Enter your choice: 2
Customer removed successfully

```

```

=== Ticket Reservation System ===
1. New Customer
2. Reserve booking
3. Cancel booking/ticket
4. Display my bookings
5. Find booking
6. View Most Booked Events
7. Modify/View customer
0. Exit

Enter your choice : 7
Enter Customer ID: 1145
No customers available

```

VI Work distribution

<u>Name</u>	<u>Fajr</u>	<u>Shujon</u>	<u>Renad</u>	<u>Ghadi</u>	<u>Ghedaf</u>
Singly Linked List	Search()	Show Customers()	Remove Customers()	Add Customer()	_____
Queue	_____	countTicketsByEvent()	cancelTicket()	Method (Display())	findTicketBy Number()
Stack	findBookingByID()	_____	displayUser BookingHistory	Method (Display())	findMostBooked Event()
Main class	All works in main class was done together.				